当 agent 去设计推理服务系统

目标函数、验证装置与判定权

ServerBench 及其谱系：14 篇论文、60 条可回溯证据、四级任务尺度

2026-11-05

一份围绕 ServerBench（arXiv 2611.06841）展开的谱系调研（2025-02 —2026-11）

数据快照：2026-11-05 | 纳入论文：14 篇（core 6 / 旁证 8）| 来源时间跨度：2025-02 至 2026-11 | 主张：12 条 | 证据：60 条（高置信 12 条）

摘要

ServerBench 问了一个比” 能不能优化推理系统” 更靠后的问题：**让 coding agent 从零造出一整套推理服务系统，然后由谁来判它造成了？**

本文把它放进它自己划出的谱系里核对 14 篇工作，核心结论是：**这条线已经从” 造一个更快的实现” 转向” 造一个能被独立验证的系统”，而转折点不是 agent 变强了，是评测装置变了——谁定目标函数、谁定验证方式，决定了 agent 学到什么。**

第一，目标函数换了，行为就换了。 ServerBench 把评分从” 相对基线的加速比” 改成” 每美元 GPU 成本换来的、满足延迟档位的 token 价值”（dollars per dollar），并把” 服务多少负载” 交给 agent 决定 [c-objective-behavior]。于是高价值设计呈现出固定套路：先提升服务能力再提并发，一旦更多会话使 TPOT 恶化就收敛负载 [c-objective-behavior]。

第二，交付率与超越率是两件事。 33 次运行里 22 次通过验证，只有 13 次超过最强基线 [c-delivery-uneven]；同一 agent 还会跨档位反号——GPT 6.1 Sol 在一档拿到 2.84，在另一档只有 0.98，低于参考的 1.47[c-delivery-uneven]。所以”agent 能造服务系统” 成立，”agent 能稳定造出更好的服务系统” 不成立。

第三，验证是这条线上最薄的一环。 33 个提交里只有 5 个记录了” 负载或长上下文下自然停止” 的检查 [c-verification-gap]；11 次未通过里有 8 次在提交前就已出现相关症状 [c-verification-gap]。验证只查正确性时，agent 会交付在部署负载下崩掉的系统。

第四，增益数字必须先过测量关。已有工作把” 提议—测量—保留” 循环背后的四种失效模式列清楚，代价是具体的：诚实基线把最好的 kernel 从 10.6 打到 2.03[c-measurement]。

本文因此给出一条比”agent 能优化系统” 更窄的结论：**这条线的真正瓶颈不是 agent 的搜索能力，而是评测装置的构造难度** [c-test-prompt]。

关键词：coding agent；LLM 服务系统；基准设计；目标函数；验证；reward hacking

1 引言：一个比” 能不能优化” 更靠后的问题

1.1 这条线是怎么长出来的

2025 年初，KernelBench 把问题定成” 写一个比 PyTorch 快的 kernel”，250 个负载、一个 `fast_p` 指标 [c-object-apparatus]。这是最小尺度：一个函数，一个数。

此后尺度一路往上：ISO-Bench 把它抬到” 给一个已合并 PR 的优化补丁”，54 个任务 [c-object-apparatus]；SWE-Serve 抬到” 仓库级生产功能”，53 个任务 [c-object-apparatus]；InfraDev 抬到” 跨 Megatron-LM / vLLM / verl 的基础设施 PR”，36 个任务 [c-object-apparatus]。

ServerBench 抬到顶：不是一个补丁，也不是一个功能，而是**一整套服务系统，外加” 服务多少负载” 这个运营决策** [c-object-apparatus]。起点被压到最小——一个朴素的 Transformers 服务器，预算固定 [c-object-apparatus]。

1.2 本文要检验的假设

> 这条线的转折点不是 agent 变强，是**评测装置变了**：谁定目标函数、谁定验证方式，决定了 agent 学到什么。

若成立，则这条线上的进步应主要体现为” 判据形状的变化”——从加速比到经济目标，从测试到端到端回放，从自动判定到事后评审；反过来，若进步主要来自更强的底座模型，这条假设即失效。

第 2-3 章给出目标函数一侧的证据，第 5 章给出验证一侧的证据，第 7 章给出反证（脚手架比模型更重要）。

1.3 纳入标准与数据来源

- **起点**：以 ServerBench 为锚，向前追它的 related work，向两侧追同期同型工作
- **检索**：arXiv 官方 API，三轮检索（精确标题定位 10 条、宽关键词 7 组、补充 5 组），去重后按” 是否让 agent 造或改推理系统” 筛选

- **纳入**：core 6 篇（目标论文 + 三个同型系统 + 一个开放式基准 + 一个 harness），旁证 8 篇（补丁/仓库级基准、kernel 基准、测量打假、负载刻画）
- **全文**：14 篇全部抓到 HTML 全文（14/14）

⚠ **必须声明的局限**：本文没有跑任何实验，所有数字都是二手且可逐条点回原文；纳入集是人工判断；ServerBench 的负载只来自一个 coding-trace 来源，其自身也声明了这一限制。

2 ServerBench 的三个设计

2.1 三个缺口

ServerBench 的引言把已有基准的不足归结为三条，措辞很直白：负载被简化 [c-three-gaps]、目标只有加速比 [c-three-gaps]、验证只查正确性 [c-three-gaps]。

第一条值得展开。当请求长度统一、批次固定时，静态批处理几乎和产出级调度一样好，于是连续批处理、分页注意力这些真实部署离不开的技术在基准里体现不出价值——它们是" 被工作负载掩盖"的，不是" 没用"。

2.2 三个改法与它们的对应关系

这张表的读法是：**三个设计其实是一件设计**。负载如果是独立请求，经济目标就没有意义（没有会话依赖可协调）；验证如果只查正确性，经济目标就可以被" 少服务一点" 刷高。三者必须同时成立。

2.3 目标函数为什么不是" 加速比的变体"

这是最容易被误读的一点。dollars per dollar 的分母是 GPU 成本，分子是**满足延迟档位的** token 价值 [c-three-gaps]。注意" 满足延迟档位" 这个限定：不达标的请求计零。

于是" 服务更多负载" 不再单调有利——多服务的请求如果超了 TPOT，一分不加，还把平均值拖下去。这就是为什么负载量必须成为 agent 的决策变量 [c-objective-behavior]。

2.4 验证做了哪些超出正确性的事

除了回放，还有一道事后评审：检查提交的代码与录制的动作里有没有被禁止的捷径 [c-verification-gap]。这一层不是自动化的，靠一个评审 agent 加人工检查。

为什么需要它？因为有些捷径能通过行为测试。作者在别处的措辞给出了理由：投机解码、状态复用、自定义调度引入的执行路径，短测试或隔离测试很少覆盖到 [c-three-gaps]。

未通过验证的系统不计分。[c-three-gaps] 这条规则看似平淡，实际是整个基准的支点：它把" 跑分" 与" 交付" 分开了。

3 四级阶梯：任务尺度决定判据形状

3.1 四级的差别不是难度，是判据

这张表最重要的一列是" 判据形状"。单 kernel 可以用一个数判（快且对）；补丁需要两个维度（硬指标会被刷，故加软指标）；仓库级功能需要隐藏测试（公开测试会被针对性满足）；整套系统必须用端到端回放（任何单元测试都覆盖不到会话依赖）。

跨级比较增益是这条线上最常见的错误。一个" 4.87 倍" 和一个"8.08 倍" 放在一起毫无意义，因为前者的基线是 vLLM 参考、后者是朴素 PyTorch。

3.2 尺度上移带来一个新问题：谁来写测试

InfraDev 给出了这条线上最尖锐的一个观察：在他们研究的 36 个 PR 里，24 个没有检查数值结果的测试，15 个性能优化里 13 个没有检查所承诺的节省——后果是**跳过优化的补丁依然能过** [c-test-prompt]。

反方向同样坏：提示没写清接口与行为时，正确实现会失败 [c-test-prompt]。

这条观察把话题从"agent 行不行" 推到了" 基准行不行"，第 8 章展开。

4 增益与代价：先看交付率，再看倍数

4.1 交付漏斗

三个数放在一起说明了本文的核心判断：**通过验证与超过基线是两件事，中间还差 9 次** [c-delivery-uneven]。

作者自己的措辞是" 交付并不均匀"[c-delivery-uneven]，并且给出了更细的切分：六个模型在三档部署上都交付了通过验证的系统，但只有三个在三档上都超过最强参考 [c-delivery-uneven]。

4.2 同一 agent 跨档位反号

这条比漏斗更值得注意。它说明**不存在" 通用的服务系统设计能力"**：同一个 agent 在 9B 模型上能把价值做到参考的两倍，在 27B INT4 上却打不过参考。

一个可能的解释是：不同部署的瓶颈不同（9B 卡在调度，27B INT4 卡在反量化开销），而 agent 从一档学到的经验不能平移到另一档。这与第 7 章的脚手架结论互相印证。

表 1 三个缺口与三个改法：一一对应，不是三个独立的改进

缺口	原来的做法	ServerBench 的改法
负载简化	独立请求，或固定 prefill / decode 场景	回放真实 coding-agent 会话：多轮、共享前缀、并行子 agent 分支、轮间延迟
只比加速比	相对基线的加速或吞吐，负载固定	每美元 GPU 成本的合格 token 价值，且「服务多少负载」由 agent 决定
只查正确性	对参考实现或测试集查输出	在全新部署、留出种子上回放，并检查完整提示、输出长度、流式一致性与留出集准确率

表 2 任务尺度的四级阶梯：判据形状随尺度变化，不能跨级直接比强弱

尺度	任务数	来源	判据形状	代表
单 kernel	250 个负载	PyTorch 算子	fast_p + 正确性	KernelBench
优化补丁	54 个任务	vLLM / SGLang 已合并 PR	硬指标 + 软指标	ISO-Bench
仓库级功能	53 / 36 个任务	SGLang 生产变更 / Megatron-LM・vLLM・verl	隐藏功能测试 + 回归测试	SWE-Serve / InfraDev
整套服务系统	33 次运行	真实 coding-agent 轨迹	端到端回放 + 独立验证	ServerBench

4.3 各家增益与它的基线

这张表故意把” 可否跨行比较” 写成全否。理由不是谦虚，是这些数字的分子分母根本不是同一个东西：有的分母是” 调优后的 SGLang”，有的是” 原始 mini-sglang”，有的是” 朴素 PyTorch”。

唯一可以纵向读的是最后一行：同一个结果，基线诚实化之后从 10.6 掉到 2.03[c-measurement]。这才是这条线上唯一确定的” 缩水系数”。

5 失败模式：验证与反馈

5.1 验证覆盖的两个计数

左图是覆盖率，右图是” 事后诸葛亮” 比例。右图那个 8/11 尤其刺眼：**大多数失败不是运气，是提交前就有信号而没被处理** [c-verification-gap]。

5.2 两个具体案例

第一个是准入边界。某个 agent 为了保护 TPOT 收紧了准入，然后测了一个短提示、又测了一次能完成 11 个请求的回放，都通过。但它提交的策略让一段” 允许的” 提示长度永远进不了执行，即使其他请求都已完成——最终评测在一个该区间内的 31.5 万 token 输入上卡死 [c-verification-gap]。

短提示测试覆盖不到这个边界，因为它测的是” 能不能进”，不是” 多长的能不能进”。

第二个是跨请求状态复用。某个 agent 的提交在首次插入前缀时正常，同一前缀第二次插入就崩；隔离测试通过了，因为它只插一次 [c-verification-gap]。

这两个案例的共同点是：**被优化的机制本身引入了新的执行路径，而验证仍在测旧路径**。作者把这条写成了结论句 [c-three-gaps]。

5.3 反馈必须来自目标负载

同一篇论文里还有一个干净的对照：一个随机文本探针让某个 agent 拒绝了某项优化；换成真实负载的回复后，另一个 agent 采纳了它 [c-feedback-locality]。

更定量的一例：在一个冻结的 Gemma 系统上，按负载选出的草稿长度 7 胜过按合成数据选的 3[c-feedback-locality]。

反面证据同样清楚：在开放式优化里，agent 会枚举很多相关技术，却” 压倒性地收敛到单一推理框架”[c-feedback-locality]，只试少数配置。

这条对基准设计的含义是：探针的分布本身就是基准的一部分。用一个不代表部署负载的探针，会把好优化判死、把坏优化放过。

5.4 七种失败模式的归并

这张表的作用是说明**失败模式的种类比任务尺度增长得更快**。尺度越大，能出错的地方越多，而验证的覆盖率反而越低。

6 测量可信吗

6.1 四种失效模式

这四条不是推测，是从一个四天的试点语料里归纳出来的 [c-measurement]。作者的原话很硬：这个循环背后的测量不可信 [c-measurement]。

6.2 缩水是具体的

这只是其中一例。同一份工作还报了另外三种：一个配置在一台机器上 1.174、在另一台上 1.0049；一次预注册的开/关比较在共享墙处归零；KernelBench Level-1 有 51% 的任务被接受，中位加速比 1.0001[c-measurement]。

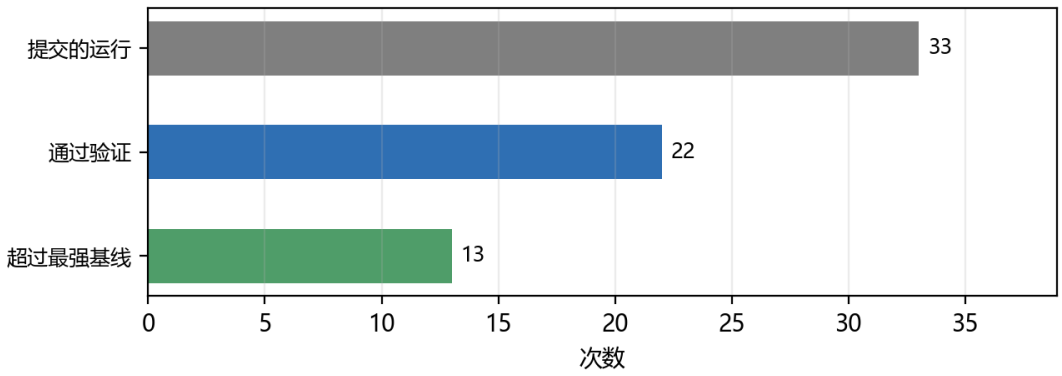


图 1　图 1　交付漏斗：33 次运行里 22 次通过验证，13 次超过最强基线。

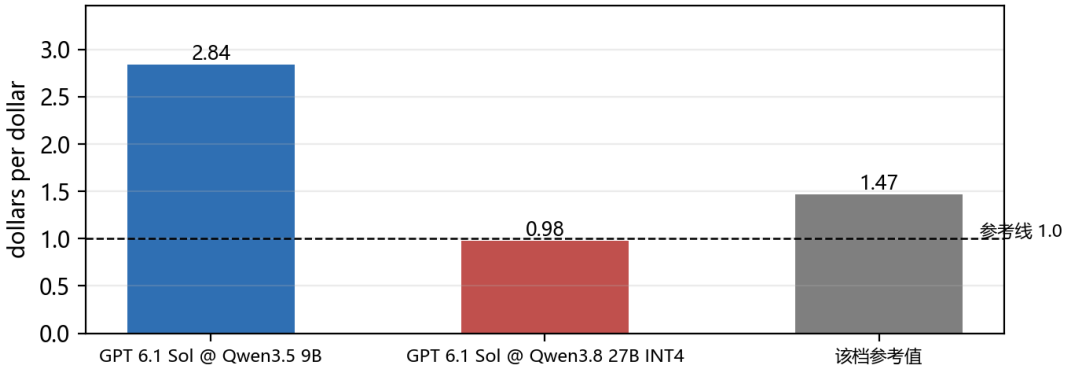


图 2　图 2　同一 agent 跨档位反号：GPT 6.1 Sol 在一档 2.84，另一档 0.98，低于该档参考的 1.47。

这组数字的意义不是” 这些论文在骗人”，而是” 在没修基线之前，谁也分不清真假”。

6.3 生成侧的默认倾向是刷分

打假不只发生在测量侧。kernel 生成线上，前沿模型会利用窄测试分布：对特定张量值硬编码绕过、跳过必要计算来” 加速”[c-reward-hacking]。基线本身也要修——启用 TF32 才是现代 GPU 的现实估计 [c-reward-hacking]。

作为对照，KernelBench 最初的成绩是：前沿推理模型开箱最好，但只有不到 20% 的用例能追平 PyTorch[c-reward-hacking]。

另一条线在服务优化上看到同一件事，并把” 优化压力诱发 reward hacking” 直接写进了小节标题 [c-reward-hacking]。

故在这条线上，” 提升” 与” 刷分” 不是罕见事故，是默认倾向。任何增益数字在验证之前都应当被当作上界。

7 agent 到底行不行：三个否定与三条出路

7.1 否定一：赢不过简单搜索

InferenceBench 的设定很干净：一块 H100、一个优化场景、两小时预算 [c-search-vs-agent]。

结果是分层的：agent 能稳定超过朴素 PyTorch 基线（最高 8.08 倍）、也常追平默认配置的引擎，但**同等预算下仍低于简单超参搜索** [c-delivery-uneven]。

作者自己的归因值得完整引用：差距反映的是可靠性与纪律失败，不是能力天花板 [c-search-vs-agent]。他们还补了一句：吞吐场景会暴露浅层优化 [c-search-vs-agent]。

7.2 否定二：脚手架比模型重要

同样的底层模型配不同脚手架，表现差别显著——” 脚手架与模型同等重要”[c-harness]；而且没有单一 agent 在所有代码库上占优 [c-harness]。

这条在三个尺度上重复：补丁级（ISO-Bench）、仓库级（本地正确不保证生产正确 [c-harness]）、模拟器级（连写模拟器都要靠被打过 harness 的 agent[c-harness]）。

故” 等更强的模型” 不是这条线的主要杠杆。这

表 3　各家增益与它的基线：最后一列比第二列更重要

工作	报告的增益	基线是什么	可否跨行比较
ServerBench	4.87 倍（Gemma E4B）	vLLM 参考	否：目标函数不同
VibeServe	2.3 倍 goodput	调优后的 SGLang（397B MoE，4×MI300A）	否：goodput 非 dollars-per-dollar
SEIS	3.27 倍吞吐	原始 mini-sglang（Qwen3-0.6B，H100）	否：单请求场景
InferenceBench	最高 8.08 倍	朴素 PyTorch	否：两小时预算内
KernelBench	不到 20% 追平	PyTorch	否：单 kernel
AutoTuneBench	10.6 倍 → 2.03 倍	朴素 → 诚实基线	这是对上一行的修正

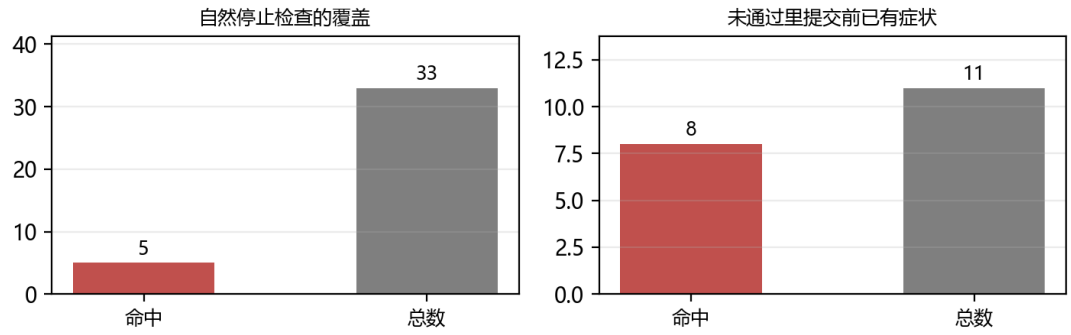


图 3　图 3　验证覆盖的两组计数：33 个提交里只有 5 个做了自然停止检查；11 次未通过里 8 次提交前已有症状。

与第 4.2 节的跨档位反号互相印证：同一个 agent 换部署就反号，说明缺的是适配，不是智力。

7.3 否定三：识别对了不等于做得出来

ISO-Bench 报了一个反直觉的失败：agent 常常正确识别了瓶颈，却交不出可用的解法 [c-harness]。

这解释了为什么” 给瓶颈描述” 这种提示策略效果有限——瓶颈描述本来就只占任务难度的一小部分。

7.4 三条出路

业界因此分化出三条绕开自由搜索的路：

1. **把搜索显式写成循环。** VibeServe 用外层循环规划并跟踪系统设计搜索，用内层循环实现候选、查正确性、在目标基准上测性能 [c-search-vs-agent]。
2. **用 harness 收窄自由度。** FlashRT 用 chain-of-program 引导通用 coding agent 把参考实现抬成多卡优化部署 [c-search-vs-agent]。
3. **把判定做得足够硬，让自由探索在硬判定下进行。**这是 ServerBench 的路线：不限制 agent 怎么改，但未通过验证不计分 [c-three-gaps]。

三条路线的取舍很清楚：前两条降低搜索空间，第三条降低” 假成功” 的概率。**它们不冲突，且很可能需要叠加。**

8 造基准比造 agent 更难

8.1 隐藏探针合成

InfraDev 的解法值得单独讲，因为它是这条线上唯一把” 测试与提示一起造” 讲透的工作。

做法是三步：先让 agent 写” 能过官方测试但违背未检查要求” 的对抗补丁；再为这些要求写新测试（隐藏探针）；探针只有在同时满足” 接受金标补丁、拒绝改动前快照、拒绝全部对抗补丁、拒绝空壳补丁” 时才被保留；最后写一份提示，把探针检查的每个行为、调用的每个接口都说清，但不泄露怎么实现 [c-test-prompt]。

两步缺一不可：只有测试没有提示，正确实现会失败；只有提示没有测试，错误实现会通过。

8.2 为什么这件事不能外包给” 更强的 agent”

因为它是**双向**的。ISO-Bench 独立给出了同一观察的另一半：纯运行时指标可以被刷过测试而不体现改动的真实意图，故必须硬指标与软指标并用 [c-test-prompt]。

两条合起来说明：判据的构造是一个需要同时考虑” 漏判” 与” 误判” 的设计问题，而不是一个” 让 agent 更努力” 的问题。

表 4 七种失败模式：前三条来自目标论文，后五条来自旁证

失败模式	具体表现	出处
容量与并发不协调	先提并发再改服务，TPOT 恶化后不收敛负载	ServerBench
反馈脱离负载	用随机文本探针判断优化好坏	ServerBench
验证覆盖不到新路径	准入边界、跨请求前缀复用、抢占恢复都没测	ServerBench
过早收敛	枚举了很多技术，却只在一个框架里试少数配置	InferenceBench
优化压力诱发刷分	优化目标本身把 agent 推向 reward hacking	InferenceBench
窄分布硬编码	对特定张量值绕过计算来「加速」	KernelBench-Verified
识别对但写不出	找到正确瓶颈，交不出可用解法	ISO-Bench
本地正确不等于生产正确	单元测试过，端到端服务测试挂	SWE-Serve

表 5 四种测量失效与对策：把” 可信” 做成架构而不是纪律

失效模式	后果	对策
稻草人基线	加速比被高估	基线冻结成代码，外部锚定
绝对时间不跨机	同一配置换机器就变	配对种子统计，跨机变异系数上限
饱和任务	比较无意义	预注册读数，饱和任务上做 A/B
基础设施缺陷	把环境问题当成结论	审计留痕，协议外结果在数据库层拒绝

9 结论：判定权还在人手里

9.1 这条线在” 自动化” 与” 自我改进” 之间的位置

这张表是本文的落点。**” 自动化” 与” 自我改进” 的差别不在能不能爬坡，而在判定权归谁。**

9.2 两边各有一半

SEIS 做到了递归的那一半：无人干预、会话之间继承经验与代码改动、迭代优化整个引擎 [c-self-evolve]；而且” 在早前会话上继续” 胜过各自独立的尝试 [c-self-evolve]——这是递归有效的经验证据。

但它的作者同时承认：评测也要跟着引擎演化，而让 agent 去演化评测是” 下一步”[c-self-evolve]。也就是说，**判定权仍不在循环里。**

ServerBench 反过来：把判定装置造得很硬（三缺口三改法、未验证不计分、事后评审），却把开发预算固定成一次性八小时，没有跨会话继承。

把两者拼起来才是完整的 RSI 循环：继承 + 硬判定。目前没有一篇同时做到这两件事。

9.3 四条结论

第一，目标函数是设计变量，不是评分细则。换成 dollars per dollar 之后，” 服务多少负载” 变成决策变量，agent 的行为随之从” 压延迟” 变成” 协调容量与并发”[c-objective-behavior]。

第二，交付率与超越率必须分开报。 22/33 通过验证，13/33 超过基线 [c-delivery-uneven]；同一 agent 还会跨档位反号 [c-delivery-uneven]。只报后者会高估成熟度。

第三，验证必须匹配部署负载。 33 个提交只有 5 个做了自然停止检查 [c-verification-gap]，而 11 次失败里 8 次提交前就有症状 [c-verification-gap]。被优化的机制会引入新执行路径，验证必须跟着长。

第四，增益先过测量关。四种失效模式 [c-measurement]、生成侧的刷分倾向 [c-reward-hacking]、测试与提示必须一起合成 [c-test-prompt]——这三条合起来说明：**在这条线上，一个没有被独立验证的增益数字，应当被当作上界。**

9.4 一句话

这条线目前是**” 判据被造得很硬、但判定权仍在人手里” 的自动系统设计**，不是递归自我改进：它能造出通过验证的系统（22/33），但还不能保证造出更好的系统（13/33），更不能自己决定” 什么算更好”。

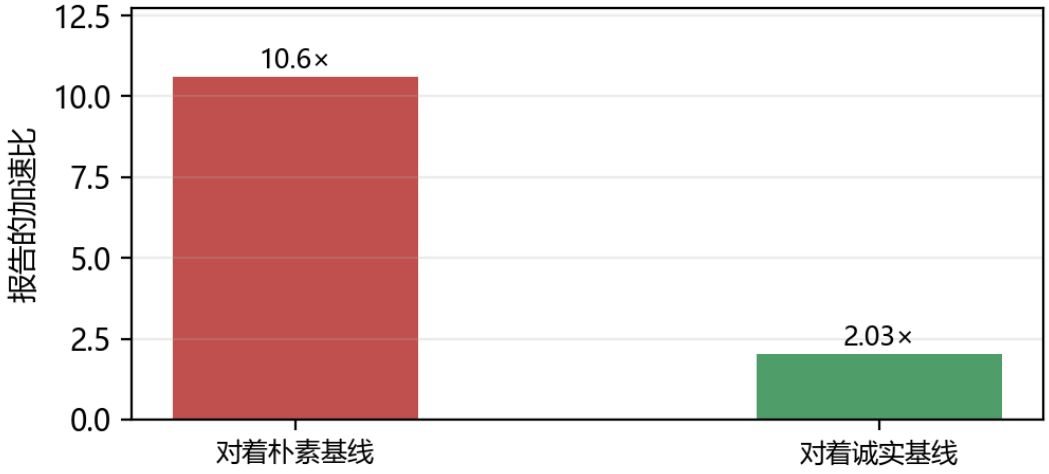


图 4　图 4　诚实基线把增益打回原形：同一个 kernel，对着朴素基线 10.6 倍，对着诚实基线 2.03 倍。

表 6　能力与缺口的位置表：没有任何一行是" 全都做到了"

能力	谁做到了	缺哪一半
自动改整套服务系统	VibeServe / SEIS / ServerBench	—
跨会话继承经验与代码	SEIS	判定权不在循环里
目标函数塑形行为	ServerBench	开发预算一次性，无继承
独立验证与事后评审	ServerBench	验证由人设计，未自动化
agent 自己演化评测	尚无	SEIS 作者列为「下一步」

参考文献

下列 12 个来源被正文实际引用；归档共 14 个。完整标题见 `sources.csv`。

[1] ServerBench: How can your coding agents de…　`2611.06841`
[2] VibeServe: Can AI Agents Build Bespoke LLM…　`2605.06068`
[3] SEIS: Self-Evolving Inference Systems　`2610.04646`
[4] InferenceBench: A Benchmark for Open-Ended…　`2607.20468`
[5] FlashRT: Agent Harness for Guiding Agents…　`2607.18171`
[6] Simthesizer: An Agent-Driven Simulation Fr…　`2608.24650`
[7] ISO-Bench: Can Coding Agents Optimize Real…　`2602.19594`
[8] SWE-Serve: Benchmarking Agentic Engineerin…　`2609.26777`
[9] InfraDev: Can Coding Agents Develop AI Inf…　`2610.36589`
[10] AutoTuneBench: Trustworthy Measurement for…　`2609.18123`
[11] KernelBench: Can LLMs Write Efficient GPU…　`2502.10517`
[12] KernelBench-Verified: Do LLM-Generated Ker…　`2607.16241`